
CSS PHY PROJECT | TECHNICAL DEEP DIVE

The CSS PHY Transmitter Explained From Zero

*A complete conceptual map of the IEEE 802.15.4
Chirp Spread Spectrum (CSS) 2450 MHz PHY transmitter*

Covers the packet structure, the full processing pipeline block by block, the chirp-generation mathematics, and a fully worked numerical example — everything needed before opening MATLAB.

CSS PHY Project

Reference document — read before implementation

Contents

The CSS PHY Transmitter, Explained From Zero	1
1. What you're building, in one paragraph	1
2. The packet: what actually gets sent	1
2.1 The SFD (Start-of-Frame Delimiter)	2
2.2 The PHR (PHY Header)	2
2.3 The PSDU (payload)	2
3. The transmitter pipeline: block by block, in order	2
4. Block 1 – Zero Padding	3
5. Block 2 – Demux (I/Q split)	4
6. Block 3 – Serial-to-Parallel + Symbol Mapper	4
6.1 Serial-to-Parallel	4
6.2 Symbol Mapper (the bi-orthogonal codeword lookup)	4
7. Block 4 – Bit Interleaver (250 kb/s only)	5
8. Block 5 – Parallel-to-Serial + QPSK Mapper	5
8.1 Parallel-to-Serial	5
8.2 QPSK Mapper	5
9. Block 6 – DQPSK Coding	5
10. Block 7 – CSK Generator (the chirp sequence itself)	6
10.1 What gets generated	6
10.2 The subchirp equation	6
10.3 The sign tables	6
10.4 The raised cosine window	7
10.5 Timing constants	7
11. Block 8 – Chirp Modulation (DQCSK)	7
12. Fixed-point representation (why there's a "DAC bit number")	8
13. Putting it all together – the full worked example	8
14. Quick-reference: which MATLAB file implements which block	9
15. Quick-reference: every global variable you'll see, and what it means	9

The CSS PHY Transmitter, Explained From Zero

This document assumes you know nothing about this specific project yet, and walks through every piece of the CSS PHY transmitter in detail – what it is, why each block exists, exactly what math it does, and what numbers you should expect to see once you start running the MATLAB code. Read this before you touch MATLAB. The prerequisites guide covered general communications background; this document covers the actual project you're about to build.

1. What you're building, in one paragraph

You're building the **transmitter half** of a Chirp Spread Spectrum (CSS) physical layer radio, as defined by the IEEE 802.15.4 standard's 2450 MHz alternate PHY. Concretely: it's a chain of processing blocks that takes a string of payload bytes (e.g. "send these 25 bytes to the other device") and turns them into a stream of complex I/Q samples representing a spread-spectrum chirp waveform, ready to feed a digital-to-analog converter and an antenna. Nothing about receiving is in scope here – this is transmit-only.

2. The packet: what actually gets sent

Every transmission is one **PPDU** (PHY Protocol Data Unit). It has three parts, always in this order:

1. **Synchronization header** – a Preamble followed by a Start-of-Frame Delimiter (SFD)
2. **PHY Header (PHR)** – says how many payload bytes follow
3. **PSDU (payload)** – your actual data bytes

The exact sizes depend on which of the two CSS data rates you're using:

Field	1 Mb/s	250 kb/s
Preamble	32 chirp chips (all +1)	80 chirp chips (all +1)
SFD	16 chips	16 chips
PHR	12 bits (4 chirp symbols)	12 bits (2 chirp symbols)
PSDU	up to 127 bytes	up to 127 bytes

Notice “chip” and “symbol” both appear – a symbol at the PHY level here isn't 1 bit, it's a whole chirp sequence (explained in Section 7). The preamble is just a long run of a known, fixed pattern – its entire purpose is to let a receiver lock onto timing before anything meaningful arrives. You (as the transmitter) don't need a receiver to generate it: it's always the same fixed sequence.

2.1 The SFD (Start-of-Frame Delimiter)

The SFD marks the exact boundary between “still preamble” and “here comes the header.” It's a specific fixed 16-bit pattern (different for each data rate), applied directly to both the I and Q inputs of the QPSK mapper. From the actual project code (`globalSettings.m`), the two patterns are:

- **1 Mb/s SFD:** -1 1 1 1 -1 1 -1 -1 1 -1 -1 1 1 1 -1 -1
- **250 kb/s SFD:** -1 1 1 1 1 -1 1 -1 -1 -1 1 -1 -1 -1 1 1

These are bipolar (+1/-1) values, not 0/1 bits – that's because they get fed straight into the QPSK mapper, which expects +1/-1 inputs (Section 7.5). When you print them as binary text files (as the code does), -1 maps to 0 and +1 maps to 1.

2.2 The PHR (PHY Header)

12 bits, laid out as:

Bits	Meaning
0-6 (7 bits)	Payload length, in bytes (0-127)
7-8 (2 bits)	Not used (always 0)
9-11 (3 bits)	Reserved (always 0)

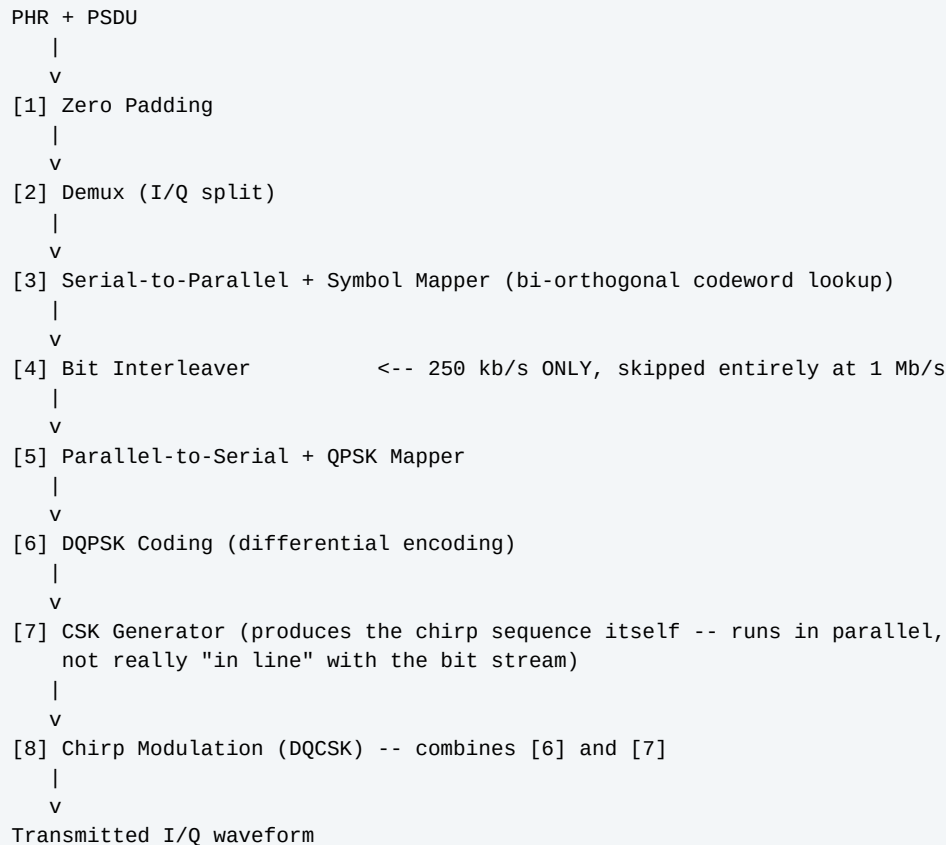
So if your payload is 25 bytes, the PHR's first 7 bits encode the number 25 in binary, and the remaining 5 bits are just zero-padding.

2.3 The PSDU (payload)

This is just your data bytes, up to 127 of them. Nothing clever happens to them individually before the pipeline starts – they're concatenated right after the PHR and fed into the rest of the chain as one long bitstream (`binaryData = [PHR, incomingStream]` in the code).

3. The transmitter pipeline: block by block, in order

Here is the complete chain, in the exact order data flows through it. This mirrors Figure 2-1 in your thesis document and the actual function-call order in `ChirpSpreadSpectrum_Tx.m`.



Separately and *before* any of this, the **Preamble + SFD** (Section 2.1) is generated and simply glued onto the front of the I and Q paths right before the QPSK mapping step – it doesn't go through zero-padding, demux, or symbol mapping, because it isn't "data" in the coded sense; it's a fixed known pattern applied directly.

Now, block by block, in detail.

4. Block 1 – Zero Padding

Purpose: the blocks downstream (Symbol Mapper, Interleaver) consume bits in fixed-size groups. If your PHR+payload bit count isn't an exact multiple of that group size, you pad with zero bits at the end until it is.

The exact rule:

- At 1 Mb/s: pad until the total is a multiple of **6** bits (because I and Q paths each take 3 bits per symbol, 2 paths x 3 = 6)
- At 250 kb/s: pad until the total is a multiple of **24** bits (2 paths x 6 bits x 2 consecutive codewords needed by the interleaver = 24)

Worked example (this is the default configuration in the project: 25-byte payload, 1 Mb/s):

```

payload bits = 25 x 8 = 200
PHR bits     = 12
total        = 212
212 mod 6    = 2

```

```
padding      = 6 - 2 = 4 zero bits added
padded total = 216 bits
```

You'll see exactly this arithmetic reflected in the MATLAB variable `paddingBy` once you run the code.

5. Block 2 – Demux (I/Q split)

Purpose: CSS, like the QPSK modulation it's built on, transmits two independent bit streams simultaneously – one on the I (in-phase) carrier component, one on Q (quadrature). This block is simply where the single padded bitstream gets split into those two streams.

The exact rule: alternate bits, starting with I.

```
bit 1  -> I
bit 2  -> Q
bit 3  -> I
bit 4  -> Q
... and so on
```

So for a 216-bit padded stream, I gets bits {1,3,5,...} (108 bits) and Q gets bits {2,4,6,...} (108 bits).

6. Block 3 – Serial-to-Parallel + Symbol Mapper

This is really two tightly-linked steps.

6.1 Serial-to-Parallel

Each of the I and Q bit streams (108 bits each, in our running example) gets chopped into fixed-size groups:

- **1 Mb/s:** groups of 3 bits – $108/3 = 36$ groups per path
- **250 kb/s:** groups of 6 bits

Each group of bits is then converted from binary to a decimal index (0-7 for 3-bit groups, 0-63 for 6-bit groups).

6.2 Symbol Mapper (the bi-orthogonal codeword lookup)

This is the error-control coding step – each small group of bits is looked up in a table and replaced by a longer, fixed codeword. This is a **block code**, specifically a **bi-orthogonal code**, built from **Walsh-Hadamard matrices**.

Why Walsh-Hadamard specifically? A Hadamard matrix is a square matrix of +1/-1 entries where every row is exactly orthogonal to every other row (their dot product is zero). That property is exactly what you want for a codeword table: the further apart (in a correlation sense) your codewords are from each other, the more noise a receiver can tolerate before mistaking one codeword for another. “Bi-orthogonal” means the table is the Hadamard matrix rows *plus* their negations, doubling the table size while keeping strong separation between entries.

Concretely, for 1 Mb/s: a 4x4 Hadamard matrix gives 4 codewords of length 4; stacking the negated versions underneath gives 8 codewords total, each 4 chips long. 3 input bits select one of these 8 codewords ($2^3 = 8$ – exactly matches). This is a rate-3/4 code (3 bits in, 4 chips out).

For 250 kb/s: a 32x32 Hadamard matrix gives 32 codewords of length 32; with negations, 64 codewords total. 6 input bits select one of 64 ($2^6 = 64$). This is a rate-6/32 (= 3/16) code – much heavier redundancy,

which is exactly the “lower/coded rate, more robust” option the standard describes.

What you’ll actually see: `globalSettings.m` builds these tables with a single line, `hadamard(4)` or `hadamard(32)`, then stacks the negated copy underneath. When you inspect `codeword_1Mbs` in your workspace, you’ll see an 8x4 matrix of +1/-1 values – that’s the entire codeword table for 1 Mb/s, sitting right there.

7. Block 4 – Bit Interleaver (250 kb/s only)

Skip this entirely if you’re running the default 1 Mb/s configuration – the code branches around it with `if dataRate == 1`.

Purpose (when it does apply): protect against *burst* errors. If interference wipes out several consecutive transmitted chips, and those chips all belonged to the same original codeword, the whole codeword may become unrecoverable. Interleaving scrambles the chip order (in a fixed, known, reversible way) across *two consecutive* 32-chip codewords, so that a burst of consecutive corrupted chips ends up hitting many different codewords lightly rather than one codeword heavily – which the error-correcting code handles far better.

Mechanically: two consecutive 32-chip codewords (64 chips total) are permuted according to a fixed lookup table baked into `bitInterleaver.m`. It’s a pure re-ordering – no bits are changed, only their positions.

8. Block 5 – Parallel-to-Serial + QPSK Mapper

8.1 Parallel-to-Serial

The codewords produced by the Symbol Mapper (one per group of input bits) get concatenated back into one long serial chip stream per path – and this is also where the fixed Preamble+SFD pattern (Section 2.1) gets glued onto the front of each path.

8.2 QPSK Mapper

Each I chip and its corresponding Q chip get combined into a single complex number – a QPSK symbol. The four possible combinations of I,Q in {+1,-1} map to the four points of a QPSK constellation. In the actual code this is done with one compact formula:

$$\text{QPSK_symbol} = ((I + Q) - j*(I - Q)) / 2$$

Check it by hand for I=+1, Q=+1: $((1+1) - j*(1-1))/2 = (2 - 0)/2 = 1$ – a real-axis point. For I=+1, Q=-1: $((1-1) - j*(1+1))/2 = (0 - 2j)/2 = -j$ – an imaginary-axis point. You get one of exactly four unit-magnitude complex values, matching Figure 2-1’s constellation diagram from your thesis document.

9. Block 6 – DQPSK Coding

Purpose: instead of transmitting the *absolute* QPSK phase of each symbol, transmit the *change* in phase relative to a symbol four positions earlier. A receiver that doesn’t have a perfect absolute phase reference (a very common real-world situation) can still reliably measure phase *differences* between received symbols.

Why “four positions earlier” and not just “the previous symbol”? Because of how CSS groups things downstream: every 4 consecutive QPSK/DQPSK symbols get mapped onto the 4 subchirps of one chirp sequence (Section 10). The differential encoding is deliberately matched to that grouping.

The equation, straight from the code and the standard:

$$S(n) = X(n) * S(n-4)$$

where $X(n)$ is the QPSK symbol from Block 5, and $S(n)$ is the DQPSK-encoded output. This is a *phase multiplication*, not addition – multiplying two unit-magnitude complex numbers adds their phases, which is exactly the “encode the phase difference” operation.

Initial condition: since the very first 4 symbols of any packet have no “4-symbols-earlier” value to reference, the standard fixes the first 4 feedback memory values to $\exp(j\pi/4)$ (a 45-degree phase reference point) – you’ll see this as `feedback_memory = ones(4,1) + 1i*ones(4,1)` in the code (note: this particular codebase skips the $1/\sqrt{2}$ normalization deliberately, since only the *phase* matters for chirp modulation, not the magnitude – there’s a comment in the code explaining exactly this).

10. Block 7 – CSK Generator (the chirp sequence itself)

This is, in the thesis document’s own words, “the key process in the CSS transmitter” – and it’s worth understanding thoroughly since it’s also the block you’ll spend the most implementation effort on later in hardware.

10.1 What gets generated

One **chirp sequence**, selected by an index m (1, 2, 3, or 4) – there are exactly 4 possible chirp sequences defined by the standard, and your default project configuration uses `chirpIndex = 1`. Each chirp sequence is made of **4 subchirps**, indexed $k = 0, 1, 2, 3$, each one a short linear-frequency-modulated (LFM) pulse.

10.2 The subchirp equation

Each subchirp is generated from this formula (equation 1a in the standard, implemented directly in `chirpSequenceGenerator.m`):

$$\text{subchirp}(t) = \exp\left(j * 2\pi / f_s * \left(f(k,m) + \mu * \xi(k,m) / 2 / f_s * t \right) * t\right) * \text{raisedCosine}(t)$$

Breaking this down:

- $f(k,m)$ – the center frequency of this particular subchirp, in MHz. It’s always ± 3.15 MHz relative to the carrier, and *which* sign depends on both k and m (see table below).
- $\xi(k,m)$ (called `zeta_k_m` in the code) – either $+1$ or -1 , controlling whether frequency sweeps *up* or *down* over the subchirp’s duration. This is what makes it a “chirp” rather than a plain sinusoid: the t term multiplying ξ inside the frequency argument, combined with the outer $*t$, produces a t^2 term in the overall phase – and a squared-time phase term means linearly increasing/decreasing instantaneous frequency.
- μ – a fixed constant, 7.3158 (units: $1/\text{microsecond}^2$), that sets how fast the frequency sweeps.
- $\text{raisedCosine}(t)$ – a smooth tapering window (Section 10.4) applied to keep the pulse’s spectral energy contained.

10.3 The sign tables

Both $f(k,m)$ ’s sign and $\xi(k,m)$ depend on the chosen chirp index m and subchirp position k , per fixed tables in the standard:

Sign of $f(k,m)$ (rows = m , columns = $k = 1, 2, 3, 4$):

- $m=1$: - + + -
- $m=2$: + - - +

- m=3: - + + -
- m=4: + - - +

Value of $\text{xi}(k,m)$ (rows = m, columns = k = 1,2,3,4):

- m=1: +1 +1 -1 -1
- m=2: +1 -1 +1 -1
- m=3: -1 -1 +1 +1
- m=4: -1 +1 -1 +1

For the project default, $m = 1$: subchirp 0 sweeps down starting at -3.15 MHz, subchirp 1 sweeps down ($\text{xi}=+1$ means... check the code's sign convention) starting at +3.15 MHz, and so on. Rather than memorize this, the best move once you're running the code is to plot the instantaneous frequency (the chirp generator function does this automatically) and watch the 4 segments sweep – seeing it beats memorizing the table.

10.4 The raised cosine window

Purpose: an abruptly-starting, abruptly-stopping pulse has a very wide, “splattered” frequency spectrum (a sharp edge in time corresponds to a huge amount of spread-out frequency content, by basic Fourier theory). Tapering the pulse's edges smoothly instead keeps the transmitted energy contained within the intended channel bandwidth.

The shape: flat at amplitude 1 for the middle 60% of the subchirp's duration, then a smooth cosine-shaped roll-off to zero at each edge (roll-off factor 0.25). You'll see this plotted automatically the first time you run `chirpSequenceGenerator.m` – look for the trapezoid-with-rounded-shoulders shape.

10.5 Timing constants

These come straight from `globalSettings.m` and you'll see them in your workspace immediately:

Constant	Value	Meaning
Tchirp	192 samples (6 microseconds)	duration of one full chirp sequence (4 subchirps + gap)
Tsub	38 samples (1.1875 microseconds)	duration of one subchirp
Tgap	depends on m and even/odd position	the pause between chirp sequences
samplingFreqMhz	32 MHz	the assumed sample rate throughout

Note $4 \times T_{\text{sub}} = 152$ samples for the 4 subchirps, and $T_{\text{chirp}} - 4 \times T_{\text{sub}} = 40$ samples left over for the gap – split between $T_{\text{gap}}(1)$ (even chirp sequences) and $T_{\text{gap}}(2)$ (odd ones), with $T_{\text{gap}}(1) + T_{\text{gap}}(2)$ always equal to that same 40-sample budget. For `chirpIndex 1`, you'll see $T_{\text{gap}} = [10 \ 70]$.

11. Block 8 – Chirp Modulation (DQCSK)

Purpose: combine the DQPSK symbol stream (Block 6, carrying your actual data) with the chirp sequence (Block 7, generated once and reused) to produce the final transmitted waveform.

Mechanically: DQPSK symbols are consumed 4 at a time. Each group of 4 successive DQPSK symbols multiplies, one-for-one, the 4 subchirps of one chirp sequence – subchirp $k=0$ gets multiplied by DQPSK symbol n , subchirp $k=1$ by symbol $n+1$, and so on. The result: each subchirp keeps the chirp's frequency-sweeping shape, but rotated by the DQPSK symbol's phase (the “DQCSK” – differential quadrature chirp-shift-keying – effect described in the standard).

After each group of 4 modulated subchirps, a time gap is inserted – alternating between the even-position gap and odd-position gap from τ_{gap} . This alternation is deliberate: it's a standard-mandated trick to make the 4 defined chirp sequences (across different m values) even closer to mutually orthogonal, which helps multiple nearby CSS networks coexist without colliding.

What comes out: a single long complex array – your transmitted I/Q waveform, `TxchirpSequences` in the code. Its length is entirely predictable from the bit-counting arithmetic in Sections 4-9: (number of QPSK symbols / 4) * `Tchirp` samples. For the project's default 25-byte, 1 Mb/s configuration, that works out to 9216 samples.

12. Fixed-point representation (why there's a "DAC bit number")

Everything above is described in ideal, infinite-precision (floating point) math. But the actual transmitter – eventually implemented in an FPGA – can only represent samples with a finite number of bits, because that's what a real digital-to-analog converter (DAC) does. The project's floating-point MATLAB model includes one extra step to simulate this: after generating the chirp sequence, each sample is scaled and rounded down (floor) to fit into a **6-bit signed integer** (`TxDACbitNumber` = 6, giving a range of -32 to +31).

Why 6 bits specifically? The thesis document works this out by sweeping the DAC resolution from 2 to 8 bits and measuring the Mean Square Error between the fixed-point and floating-point chirp samples (covered in the prerequisites document, Section 6). 6 bits comfortably clears the standard's accuracy requirement while keeping hardware cost low – this is a number you'll see hard-coded in the MATLAB globals, not something you need to re-derive yourself, but now you know where it came from.

13. Putting it all together – the full worked example

Using the project's actual default parameters (`payloadLength` = 25, `chirpIndex` = 1, `dataRate` = 0 for 1 Mb/s), here is every number, start to finish, that you will be able to independently confirm once you run the MATLAB code:

```

payload bits           = 25 x 8           = 200
+ PHR bits             = 12
= total before padding = 212
padding added (Block 1) = 4
= total after padding  = 216 bits

split I/Q (Block 2)    -> 108 bits I, 108 bits Q

grouped into 3-bit symbols (Block 3)
-> 108 / 3 = 36 codewords per path
-> each codeword is 4 chips (rate 3/4 code)
-> 36 x 4 = 144 chips per path

no interleaving at 1 Mb/s (Block 4 skipped)

add preamble+SFD (Block 5)
-> 32 (preamble) + 16 (SFD) = 48 chips, prepended to each path
-> total per path = 48 + 144 = 192 chips
-> paired into 192 QPSK/DQPSK symbols

DQPSK coding (Block 6) -- same count, 192 symbols, now differentially encoded

chirp modulation (Block 8)
```

```
-> 192 symbols / 4 per chirp sequence = 48 chirp sequences
-> 48 x Tchirp (192 samples each)      = 9216 samples
```

```
FINAL OUTPUT: 9216 complex I/Q samples
```

Every number in this chain is something you can print to the screen and check by hand once you're running the actual code – that's exactly what the step-by-step build guide walks you through.

14. Quick-reference: which MATLAB file implements which block

Block	File
Standard tables, timing constants (Sections 6, 10.5) Preamble + SFD (Section 2.1)	common/globalSettings.m inline, top of transmitter/ChirpSpreadSpectrum_Tx.m transmitter/ChirpSpreadSpectrum_Tx.m
Zero Padding, Demux, Serial-to-Parallel, Symbol Mapper, Parallel-to-Serial, QPSK Mapper, DQPSK Coding (Sections 4-9) Bit Interleaver (Section 7)	common/bitInterleaver.m (called from within ChirpSpreadSpectrum_Tx.m)
CSK Generator / chirp sequence (Section 10)	common/chirpSequenceGenerator.m, using common/raisedCosineGen.m for the window
Chirp Modulation / DQCSK (Section 11)	transmitter/chirpModulation.m (called from within ChirpSpreadSpectrum_Tx.m)
Binary/decimal helpers used by Symbol Mapper	common/binary2decimal.m, common/decimal2binary.m

15. Quick-reference: every global variable you'll see, and what it means

Variable	Meaning
chirpIndex	which of the 4 chirp sequences (1-4)
payloadLength	payload size in bytes (1-127)
dataRate	0 = 1 Mb/s, 1 = 250 kb/s
samplingFreqMhz	assumed sample rate, 32 MHz
carrierFreqGHz	RF carrier frequency, 2.45 GHz (not used in baseband simulation, informational)
preambleLengthStd	[32, 80] – preamble length per data rate
SFDlength	16 – SFD length in chips, both data rates
PHRlength	12 – PHR length in bits, both data rates
SFD_Std	the two fixed SFD bit patterns (Section 2.1)
codewordLengthStd	[4, 32] – codeword length in chips per data rate
numBitsPerCodewordStd	[3, 6] – input bits per codeword per data rate
codeword_1Mbs	the 8x4 bi-orthogonal codeword table (Section 6.2)
codeword_250kbs	the 64x32 bi-orthogonal codeword table
Tchirp	192 samples – one full chirp sequence duration
Tsub	38 samples – one subchirp duration
Tgap	[even-gap, odd-gap] in samples, depends on chirpIndex

Variable	Meaning
TxDACbitNumber	6 – fixed-point DAC resolution (Section 12)

You now have the complete conceptual map of what every line of code you're about to run is actually doing, and what numbers to expect at each stage. When you sit down with the step-by-step build guide next, treat each command as a chance to confirm a specific number or table from this document against your own workspace – that's the fastest way to turn "I ran the code" into "I understand the system."